SEMANA 3 · AULA 2 · 04/11

COMO SE FAZ UMA SKILL

Seis passos na primeira metade, o Duelo das Skills na segunda

Felipe Oliveira · IA Aplicada à Produtividade no Desenvolvimento

<LAB365> SENAI

AGENDA DA AULA

19h00	Bloco 3: seis passos para uma skill (passos 1 e 2)	TEORIA
19h20	Demo ao vivo: o gatilho medido (passo 3)	DEMO
19h35	Passos 4 a 6: o agente escreve a skill ao vivo	TEORIA
19h45	Catálogos: skills.sh e SkillsMP	DEMO
19h55	Sorteio: 3 temas, 2 equipes por tema	DUELO
20h00	Intervalo	PAUSA
20h20	Duelo, fase 1: criar a skill	DISPUTA
20h55	Duelo, fase 2: testar a skill da adversária	DISPUTA
21h20	Auditoria cruzada, placar da semana, amanhã	FECHO

O DUELO DE HOJE

Uma skill por equipe

- 6 equipes, 3 temas de review, 2 equipes por tema
- quem roda a sua skill é a adversária
- 4 casos públicos por tema + casos novos documentados
- vence quem passa em mais casos com menos erro
- as equipes pontuam e auditam umas às outras

BLOCO 3: COMO SE FAZ UMA SKILL

Seis passos, do tamanho certo ao teste, com o gatilho medido no meio

SKILL É FUNÇÃO, NÃO MONÓLITO

ESTREITA DEMAIS

review-sqli

problema 10 skills para uma tarefa

efeito sobrecarga e instruções conflitantes

COERENTE

review-security

cobre injeção, auth, segredos, logs

efeito dispara com precisão e compõe bem

LARGA DEMAIS

review-all

problema 930 linhas, 40 regras

efeito não dispara com precisão; fábrica de FP

> **"Deciding what a skill should cover is like deciding what a function should do."** agentskills.io, Best practices.

SEIS PASSOS PARA UMA SKILL

PASSO 1

Tarefa

Do tamanho de uma função; faça à mão uma vez.

PASSO 2

Gotchas

Cada correção que você deu vira uma linha.

PASSO 3

Descrição

O quê + quando + o que não faz. Teste com near-miss.

PASSO 4

Corpo

Procedimento, template de saída, checklist. < 500 linhas.

PASSO 5

Scripts

Determinístico em scripts/; longo e raro em references/.

PASSO 6

Teste

validate.py, 3 rodadas, 2 ou 3 evals com e sem skill.

Regra de ouro: a skill nasce de uma tarefa real feita com correções, nunca de um pedido vazio ao modelo.

PASSOS 1 E 2: TAREFA E GOTCHAS

PASSO 1

Uma tarefa, feita à mão

tarefa	revisar auth.py e webhooks.py quanto a segurança (R07, R08)
tamanho	uma função: segurança, não 'tudo'
como	sessão sem skill; você corrige o agente

PASSO 2

Correções viram gotchas

você disse	"segredo se compara com compare_digest"
você disse	"SQL por f-string é injeção, mesmo interno"
vira	seção Gotchas + o que procurar

Com a transcrição na mão: "extraia desta sessão os erros que você cometeu e o que eu corrigi". É daí que sai a skill, não de "escreva uma skill de review".

PASSO 3: DESCRIÇÃO E GATILHO

NEGATIVO FRACO

Não testa nada

exemplo	"escreve uma função fibonacci"
exemplo	"qual a previsão do tempo?"
por quê	sem vocabulário em comum; qualquer descrição passa

vs

NEAR-MISS

Compartilha palavras, pede outra coisa

exemplo	"adiciona log de auditoria em refund()" (puxa security)
exemplo	"escreve o teste do webhook" (puxa correctness)
por quê	só uma descrição precisa não dispara aqui

Demo ao vivo: 4 skills instaladas, 6 prompts de trigger-eval.json, 3 rodadas cada. Vocês preveem antes, o agente responde depois. Receita: positivos > 0,5, near-misses < 0,5.

PASSO 4: O CORPO DA SKILL

review-security/SKILL.md

```
---
name: review-security
description: >
  Revisa codigo Python procurando injecao,
  auth fraca, segredo exposto e dado sensivel em log.
  Use quando pedirem review de seguranca,
  'tem falha nisso?', auditoria. Nao revisa desempenho.
---
## Gotchas
- Segredo se compara com compare_digest (R08)
- SQL sempre parametrizado; f-string e bug (R09)
## Procedimento
1. Leia a spec (R07 a R10)
2. Para cada entrada externa: query, token, log
3. Preencha a tabela de achados (template abaixo)
## Template de saida
| arquivo | linha | sev | req | evidencia |
```

CHECKLIST DO CORPO

O que não pode faltar

- descrição: o quê, quando, o que não faz
- gotchas do projeto (do passo 2)
- procedimento numerado
- template de saída com requisito
- o que NÃO fazer (escopo)
- menos de 500 linhas

PASSOS 5 E 6: SCRIPTS E TESTE

PASSO 5

Scripts e references

scripts/	collect_diff.sh, validate_report.py: determinístico, zero tokens
references/	cwe-top25.md, lido só se a tarefa pedir
sinal	o agente escreveu o mesmo helper duas vezes

PASSO 6

Validar e medir

frontmatter	python validate.py <pasta>
gatilho	3 rodadas, positivos e near-misses
evals	2 a 3 casos com e sem skill; um near-miss

Poda: instrução que não muda o resultado sai; assertion que passa sempre sai. Skill enxuta vence skill completa.

USAR O AGENTE PARA ESCREVER A SKILL

NÃO FUNCIONA

Pedido vazio

prompt	"escreva uma skill de code review"
sai	"handle errors appropriately"
por quê	sem gotchas, sem projeto, sem formato

vs

FUNCIONA

Pedido alimentado

prompt	"a partir desta transcrição, da spec e deste template, escreva a skill"
sai	gotchas reais, procedimento do projeto
depois	medir o gatilho e podar

skill-creator (anthropics/skills): automatiza evals, A/B cego e afinação da descrição. Mesmo princípio: entra transcrição e feedback, sai skill.

ONDE A COMUNIDADE PUBLICA SKILLS

SKILLS.SH

O npm das skills

quem	Vercel; leaderboard por instalações
instalar	npx skills add <owner>/<repo>
funciona em	Claude Code, Cursor, Codex, Copilot
publicar	repo público + PR em vercel-labs/skills

vs

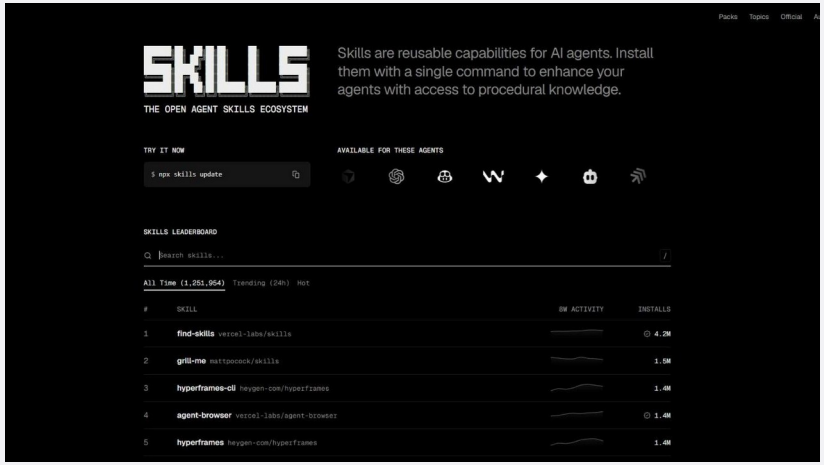
SKILLSMP

O catálogo raspado do GitHub

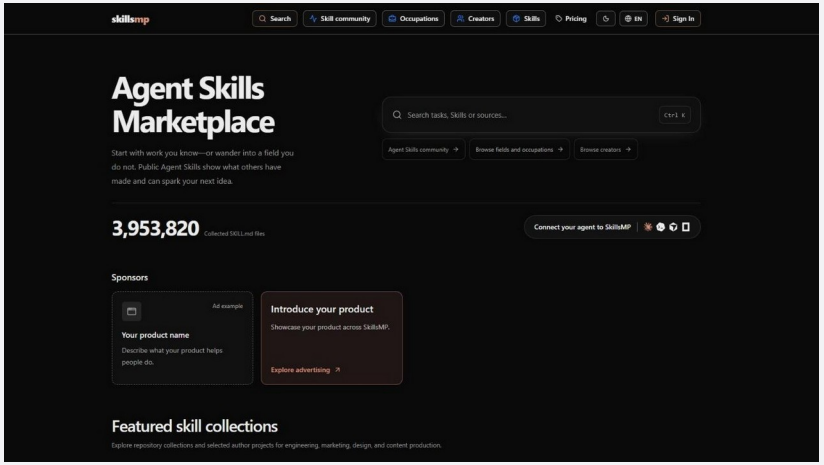
quem	projeto independente; 3,9 mi de SKILL.md
busca	por palavra ou semântica; categorias; stars
filtro	só repos com 2+ estrelas; checagem básica
aviso deles	"revise o código antes de instalar"

Antes de instalar, leia o SKILL.md: Snyk (fev/2026) achou 13,4% das skills públicas com problema crítico. Quinta vocês auditam uma dessas. Também: anthropics/skills, agentskills.io, awesome-claude-skills.

OS DOIS CATÁLOGOS, AO VIVO



skills.sh (Vercel): leaderboard por instalações; npx skills add



skillsmp.com: 3,95 mi de SKILL.md raspados do GitHub; busca semântica

Ao vivo: buscar "code review" nos dois, abrir o SKILL.md de uma skill popular e ler junto: descrição, gotchas, scripts. O que falta nela?

DISPUTA 2: O DUELO DAS SKILLS

Seis equipes, três temas, uma skill por equipe, testada pela adversária

OS TRÊS TEMAS DO SORTEIO

REVIEW-SECURITY
Injeção, auth, segredos

cobre	injeção, auth, segredo, dado sensível em log
casos	SQL por f-string; == em assinatura; PAN no log; 1 limpo
pedido	"revisa a segurança deste arquivo"

REVIEW-CORRECTNESS
Lógica, borda, estado

cobre	lógica, borda, estado, data e fuso
casos	off-by-one; datetime naive; webhook sem idempotência; 1 limpo
pedido	"revisa a correção deste arquivo"

REVIEW-PERFORMANCE
Custo e escala

cobre	complexidade, I/O repetido, memória, índice
casos	N+1; $O(n^2)$; SELECT * sem índice; 1 limpo
pedido	"revisa o desempenho deste arquivo"

Casos públicos: materiais/.../disputa-2-duelo-das-skills/casos/<tema>/. Leiam antes de escrever. Os casos novos da adversária vocês só veem na fase 2.

O DUELO: COMO FUNCIONA

PASSO 1

Sorteio

3 temas, 2 equipes por tema: security, correctness, performance.

PASSO 2

Criar (35 min)

Uma skill, pelos seis passos, no projeto da sua equipe.

PASSO 3

Trocar

A adversária instala a sua skill. SKILL.md intocado, sessão nova.

PASSO 4

Testar (25 min)

4 casos públicos, até 4 casos novos seus e 5 prompts de gatilho.

PASSO 5

Auditar

Testadora pontua na ficha, criadora confere, terceira aceita os casos novos.

Por que a adversária roda: se a skill só funciona com a criadora explicando do lado, a descrição e o procedimento falharam. É isso que o duelo mede.

FASE 1: CRIAR A SKILL (35 MIN)

PASSO 1

À mão (5 min)

Peça o review do tema no SEU projeto, sem skill. Anote cada correção.

PASSO 2

Gotchas

Cada correção vira uma linha, com o requisito da spec (R__).

PASSO 3

Escrever

template/review-tema: o quê, quando, o que não faz; procedimento; saída com linha.

PASSO 4

Scripts

grep do repetível em scripts/. Longo e raro em references/.

PASSO 5

Congelar

validate.py, 3 rodadas de gatilho, PR no repo da equipe até 20h55.

Divisão: uma pessoa faz à mão e anota, outra escreve descrição e prompts de gatilho, outra procedimento e scripts. Quem terminar primeiro escreve os near-misses.

FASE 2: TESTAR A ADVERSÁRIA (25 MIN)

PASSO 1

Instalar

Clone o repo da adversária. Copie a skill. Não edite nada.

PASSO 2

Casos públicos

4 arquivos, pedido padrão, sessão nova em cada. Saída com linha.

PASSO 3

Casos novos

Até 4 trechos seus, documentados pelo template. Rode do mesmo jeito.

PASSO 4

Gatilho

5 prompts seus: 3 devem disparar, 2 near-misses. Anote o que carregou.

PASSO 5

Entregar

Ficha + casos novos + saídas, por PR no repo da criadora, até 21h20.

Regra: quem testa não corrige a skill nem repete o pedido até sair bonito. Uma rodada por caso, a primeira resposta vale. Caso novo só pontua se a auditora aceitar que o bug é real e do tema.

PLACAR DO DUELO

EVENTO	PONTOS
caso público passado (4 por tema)	+2
caso novo passado (até 4, aceitos pela auditora)	+1
caso novo em que a skill falhou: ponto da testadora	+1
prompt de gatilho certo (5)	+1
achado sem arquivo e linha	0

Quem pontua: a testadora, na ficha, caso a caso. A criadora confere e contesta citando linha. Uma terceira equipe, de outro tema, aceita ou rejeita cada caso novo e assina o placar.

COMO VENCER

Acerto alto, erro baixo

- máximo 17 pontos por skill (8 + 4 + 5)
- vence o duelo quem pontua mais no tema
- campeã da noite: maior placar entre as 3 vencedoras
- desempate: casos novos, depois gatilho
- caso limpo: passa se nenhum achado médio+
- placar da semana por PR, assinado pelas 3 equipes

O QUE FICA E O QUE VEM

FICA

Skill nasce de uma tarefa feita à mão: gotchas, descrição medida, corpo com template, scripts, teste.

TRAGA A SKILL FUNCIONANDO

Quinta ela é a arma do Blue Team no Torneio e a base do seu trabalho final.

QUINTA (05/11)

Spec, steering e skills do SEU trabalho final; skills seguras; o Torneio Red x Blue.

LISTA DA SEMANA

Suíte completa no GitHub, evals.json + benchmark, auditoria de uma skill pública, 5 linhas do projeto. Entrega 09/11.

6

passos para uma skill

17

pontos máximos no duelo

50

min de Torneio na quinta

5

bugs que você vai plantar


<LAB365>
SENAI